Type-Directed Discretization of Probabilistic Programs

KATHERINE WU, Cornell University, USA
JULES JACOBS, Jane Street, USA
KEVIN BATZ, Cornell University, USA
ALEXANDRA SILVA, Cornell University, USA

We study *exact discretization* as a semantics-preserving transformation for recursive, higher-order probabilistic programs with continuous distributions. We target programs where continuous values are compared against finitely many constants, so exact inference reduces to a discrete problem. Our central technical contribution is a non-local, type-directed analysis that infers where continuous values can be partitioned into finitely many observationally relevant regions, then rewrites sampling and comparison behavior over those regions. We call this transformation Slice. Because this construction is global and type-directed, correctness requires reasoning beyond the local syntax: we formalize the transformation and prove soundness for boolean queries using a coupling-style logical relations argument over operational semantics ordinary operational semantics and its terminal limit, while the measurable weak simulation needed for soundness remains open. As an application, transformed programs can be executed by discrete engines such as Dice, Roulette, and Storm. Our empirical evaluation shows two complementary strengths of Slice when paired with discrete backends: it enables *exact inference* for case studies combining continuous sampling, higher-order functions, and unbounded recursion, and, on benchmarks where direct comparison is possible, it is competitive with state-of-the-art exact inference systems for continuous programs.

CCS Concepts: • **TODO**;

Additional Key Words and Phrases: probabilistic programming, exact inference, continuous distributions, discretization, type systems

ACM Reference Format:
Katherine Wu, Jules Jacobs, Kevin Batz, and Alexandra Silva. 2026. Type-Directed Discretization of Probabilistic Programs. 1, 1 (August 2026), 113 pages. https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

1 Introduction

Probabilistic programming languages (PPLs) are widely used to model uncertainty [6, 10, 14, 15, 17, 21, 24, 25, 27, 34, 35, 37, 39–41, 45–47]. Programs with *continuous* distributions capture rich models but require reasoning about uncountable domains, whereas purely *discrete* programs often admit fast and precise inference but cannot express real-valued models.

Slice studies *exact discretization*: a semantics-preserving source-to-source transformation that converts, if possible, continuous programs into discrete ones. We target higher-order recursive programs with conditioning and continuous sampling, without arithmetic. The key mechanism is *type-directed cut inference*: our type system tracks each constant threshold (or *cut*) used in comparisons and uses this information to partition continuous domains into finitely many intervals. For example, $\texttt{uniform}(0,1) < 0.5$ discretizes to $\texttt{discrete}(0.5, 0.5) < 1$, yielding a discrete program that existing exact inference tools can process.

Cuts may arise far from their source expressions: a sampled value may flow through higher-order functions, conditionals, recursive calls, data structures, and observations before it is eventually compared. To discretize soundly, Slice propagates cut information along these non-local flows, collecting all observationally relevant cut points for each continuous value and using them to partition its domain into finitely many intervals (see §2 for illustrative examples). A type system provides a natural framework for this analysis: it gives a compositional account of how cut information propagates through each language construct (via one typing

Authors' Contact Information: Katherine Wu, kaw324@cornell.edu, Cornell University, Ithaca, New York, USA; Jules Jacobs, julesjacobs@gmail.com, Jane Street, New York, New York, USA; Kevin Batz, ksb239@cornell.edu, Cornell University, Ithaca, New York, USA; Alexandra Silva, alexandra.silva@cornell.edu, Cornell University, Ithaca, New York, USA.

2026. ACM XXXX-XXXX/2026/8-ART
https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

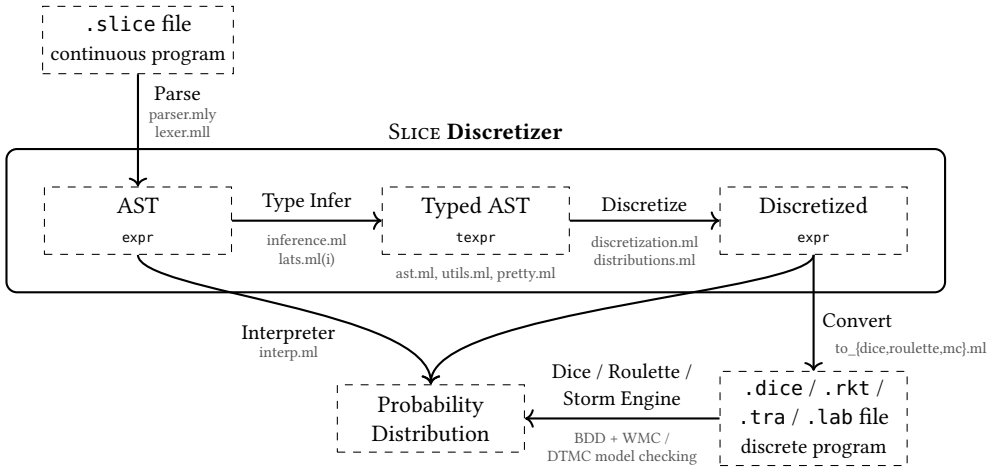


Fig. 1. Pipeline for exact discretization: typed Slice programs are transformed into semantically equivalent discrete programs, then evaluated by discrete engines. The transformation stage (type inference + discretization) is backend-agnostic; Dice, Roulette, and Storm are examples of target backends.

rule per langauge feature); it separates the analysis result, i.e. a type derivation, from the type inference algorithm used to infer it; and it supports our correctness argument, since semantic preservation can be stated as a logical relation indexed by our types carrying cut and value sets.

Our discretization method is subject to a principled restriction: a continuous value must ultimately be compared against constants; direct comparisons between two unknown reals such as $\texttt{uniform}(0,1) < \texttt{uniform}(0,1)$ cannot be discretized exactly and remain continuous. Nonetheless, benchmarks from the literature and our novel case studies show that many programs *can* be discretized by our system automatically.

Discretization itself is not new. Garg et al. [20] encode real-valued variables via user-chosen bit-precisions, trading accuracy for tractability. Other approximate discretization schemes exist in the literature [5, 12, 28]. In Slice, *exact discretization* means that the transformation is semantics-preserving: the discrete program preserves names the specification that the discrete program preserve the relevant probabilities of the original continuous program. This differs from approximate discretization, which approximates the original semantics and may provide error bounds or convergence guarantees, but does not . The ordinary terminal-limit construction is established here; the weak-simulation argument needed to prove equality of source and transformed program probabilities in the sense of our main theoremprobabilities remains open.

Figure 1 shows the concrete pipeline from typed source programs to discrete backends. Because discretization is source-to-source, any discrete PPL that supports finite distributions can serve as a backend; we instantiate this with Dice [27], Roulette [35], and Storm [16].

Outline. Starting from examples (§2) and concluding with related and future work in §8, we:

Type System (§3) Characterize when programs are discretizable.

Type Inference (§4) Infer types automatically to guide discretization.

Exact Discretization (§5) Define a type-directed transformation from continuous programs to discrete ones using inferred type information.

Soundness (§6) Prove semantic preservation of the discretization transformation with a coupling-style relational argument [7, 13, 48].

Implementation, Case Studies, and Evaluation (§7) Implement Slice type inference and the discretization transformation in OCaml, and provide Dice, Roulette, and Storm backends for running the discretized programs. We provide (i) case studies combining continuous sampling, higher-order functions, and unbounded recursion; to the best of our knowledge, there is no prior exact inference system that can cover

$e : \tau$	Condition	$\mathcal{D}[\![e : \tau]\!]$
$x : \tau$		x
$c : \texttt{float}[B; V]$	$B = \top$	c
	$B \neq \top$	$k_{\#(\lvert B\rvert+1)}$ such that $c \in \text{Intervals}(B)_k$
$\texttt{let}\ x = e_1 : \tau_1\ \texttt{in}\ e_2 : \tau_2 : \tau_2$		$\texttt{let}\ x = \mathcal{D}[\![e_1 : \tau_1]\!]\ \texttt{in}\ \mathcal{D}[\![e_2 : \tau_2]\!]$
$(\texttt{if}\ e_1 : \texttt{bool}\ \texttt{then}\ e_2 : \tau\ \texttt{else}\ e_3 : \tau) : \tau$		$\texttt{if}\ \mathcal{D}[\![e_1 : \texttt{bool}]\!]\ \texttt{then}\ \mathcal{D}[\![e_2 : \tau]\!]\ \texttt{else}\ \mathcal{D}[\![e_3 : \tau]\!]$
$(e_1 : \texttt{float}[B; V_1] < e_2 : \texttt{float}[B; V_2]) : \texttt{bool}$	$B \neq \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B; V_1]]\!] <_{\#(\lvert B\rvert+1)} \mathcal{D}[\![e_2 : \texttt{float}[B; V_2]]\!]$
	$B = \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B; V_1]]\!] < \mathcal{D}[\![e_2 : \texttt{float}[B; V_2]]\!]$
$(e_1 : \texttt{float}[B; V_1] \leq e_2 : \texttt{float}[B; V_2]) : \texttt{bool}$	$B \neq \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B; V_1]]\!] \leq_{\#(\lvert B\rvert+1)} \mathcal{D}[\![e_2 : \texttt{float}[B; V_2]]\!]$
	$B = \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B; V_1]]\!] \leq \mathcal{D}[\![e_2 : \texttt{float}[B; V_2]]\!]$
$\texttt{uniform}(e_1 : \texttt{float}[B_1; V_1],\ e_2 : \texttt{float}[B_2; V_2]) : \texttt{float}[B; V]$	$e_1, e_2 \in \mathbb{R},\ e_1 > e_2$	$\texttt{diverge} : \tau$
	$B = \top$	$\texttt{uniform}(\mathcal{D}[\![e_1 : \texttt{float}[B_1; V_1]]\!], \mathcal{D}[\![e_2 : \texttt{float}[B_2; V_2]]\!])$
	$e_1, e_2 \in \mathbb{R}$, $B = \{\sim_1 b_1, \ldots, \sim_n b_n\}$	$\texttt{discrete}(p_0, \ldots, p_n)$ where $p_k = \text{CDF}(b_{k+1}) - \text{CDF}(b_k)$
	$B \neq \top$, $B_1 \neq \top$, $B_2 \neq \top$, $V_1 = \{v_1, \ldots, v_n\}$, $V_2 = \{w_1, \ldots, w_m\}$	$\texttt{let}\ v_1' = \mathcal{D}[\![e_1 : \texttt{float}[B_1; V_1]]\!]\ \texttt{in}$ $\texttt{let}\ v_2' = \mathcal{D}[\![e_2 : \texttt{float}[B_2; V_2]]\!]\ \texttt{in}$ $\texttt{if}\ v_1' ==_{\#(\lvert B_1\rvert+1)} \mathcal{D}[\![v_1 : \texttt{float}[B_1; V_1]]\!]$ $\quad \&\&\ v_2' ==_{\#(\lvert B_2\rvert+1)} \mathcal{D}[\![w_1 : \texttt{float}[B_2; V_2]]\!]\ \texttt{then}$ $\quad\quad \mathcal{D}[\![\texttt{uniform}(v_1, w_1) : \texttt{float}[B; V]]\!]$ $\ldots$ $\texttt{else if}\ v_1' ==_{\#(\lvert B_1\rvert+1)} \mathcal{D}[\![v_i : \texttt{float}[B_1; V_1]]\!]$ $\quad \&\&\ v_2' ==_{\#(\lvert B_2\rvert+1)} \mathcal{D}[\![w_j : \texttt{float}[B_2; V_2]]\!]\ \texttt{then}$ $\quad\quad \mathcal{D}[\![\texttt{uniform}(v_i, w_j) : \texttt{float}[B; V]]\!]$ $\ldots$ $\texttt{else}\ \mathcal{D}[\![\texttt{uniform}(v_n, w_m) : \texttt{float}[B; V]]\!]$

Fig. 13. Discretization function, where $\text{CDF}(b_0) = 0$ and $\text{CDF}(b_{n+1}) = 1$. For the **uniform** distribution, $V = \top$ is implicit, and all remaining cut-set and value-set cases are ruled out by the constraints enforced during type inference. For all other language constructs, $\mathcal{D}[\![\cdot]\!]$ recurses structurally over them.

5 Discretization

This section presents local transformation rules for our type-directed discretization algorithm, with examples. Our goal is to compute, from a typed expression $e : \texttt{bool}$ and its type annotations, a *discretized* expression $\mathcal{D}[\![e : \texttt{bool}]\!] : \texttt{bool}$ such that e and $\mathcal{D}[\![e : \texttt{bool}]\!]$ are *semantically equivalent*, i.e., they induce the same distribution. We refer to this intended semantics-preserving transformation as *exact discretization*, to distinguish it from approaches [20, 28] that approximate continuous programs via externally chosen precision parameters. Exactness here is the equality proved by ??: the transformed program preserves the relevant probabilities of the source program, not merely an approximation with an error boundThe ordinary terminal-limit construction below does not prove exactness: a measurable weak simulation and its compatibility with the target terminal limit remain open obligations. Intuitively, when a program uses continuous quantities only through finitely many comparison outcomes, exact inference should reduce to a discrete problem, and exact discretization makes that reduction explicit.

As shown in Figure 13, $\mathcal{D}[\![e : \tau]\!]$ is defined recursively on $e : \tau$ and its type annotations; we focus on continuous uniform distributions, with others handled analogously. The only sub-expressions replaced by $\mathcal{D}[\![\cdot]\!]$ are float constants and sampling instructions. The rule schemata are local, but their selection and parameters come from globally inferred cut/value information; §6 proves that this non-local, type-mediated construction preserves semantics. §6 defines the intended semantic-preservation statement and identifies the weak-simulation obligations still needed to prove it. Before detailing the rules, we outline how type annotations guide discretization. The type system is designed so that every typed sub-expression of the form

$$\mathcal{D}[\![\tau]\!] = \begin{cases} \texttt{fin}(|B|+1) & \text{if } \tau = \texttt{float}[B;V] \text{ and } B \neq \top \\ \texttt{float}[B;V] & \text{if } \tau = \texttt{float}[B;V] \text{ and } B = \top \\ \text{(straightforward recursion on type structure)} & \text{otherwise} \end{cases}$$

Fig. 14. Transformation of types under $\mathcal{D}$, where $\mathcal{D}$ acts on types.

```
then 𝒟⟦uniform(0,10) : float[{<5}; T]⟧
else 𝒟⟦uniform(0,20) : float[{<5}; T]⟧ ) <#2 𝒟⟦5 : float[{<5}; {5}]⟧
```

The continuous $\texttt{uniform}(0,x)$-expression is replaced by a case distinction on the discretizations of $\texttt{uniform}(0,10)$ and $\texttt{uniform}(0,20)$, with each case executed with the correct probability 0.5. Since this reduces the remaining discretization process to discretizing expressions with *constant parameters only*, the remaining steps are completely analogous to Theorem 5.1.

Crucially, the *recoverability* constraints in the Continuous-rules from Figure 6 ensure that the parameter expressions of the respective sampling expressions can take only finitely many values, and that the cut sets are fine-grained enough to recover the values the parameter expressions evaluate to, as demonstrated in the preceding example.

A similar intuition applies to discretizable comparisons $e_1 < e_2$, where *both* expressions are non-constant but finite-cut. The *answers* constraints in the rule Less (resp. Less-Equal) ensure that at least one of e_1 and e_2 evaluates to only finitely many values, which enables discretization:

Example 5.3. Consider the following typed expression $(e_1 \leq e_2) : \texttt{bool}$, given by

```
(if discrete(0.5, 0.5) <#2 1#2 then
       uniform(0,10) : float[B; T] else 2.5 : float[B; {2.5}])
<=
(if discrete(0.5, 0.5) <#2 1#2
       then 2.5 : float[B; {2.5}] else 3.5 : float[B; {3.5}])
```

where $B = \{\leq 2.5, \leq 3.5\}$, and where e_1 : `float[B; T]` and e_2 : `float[B; {2.5,3.5}]`. Intuitively, e_1 samples from a mixture of a uniform and a dirac distribution, and e_2 samples from a finite distribution. Why can $e_1 \leq e_2$ be discretized even though neither e_1 nor e_2 is a constant? As suggested by its type, e_2 can take only finitely many values, namely $\{2.5, 3.5\}$. We apply $\mathcal{D}[\![\cdot]\!]$:

```
(if discrete(0.5, 0.5) <#2 1#2 then discrete(0.25, 0.1, 0.65) else 0#3)
<=
(if discrete(0.5, 0.5) <#2 1#2 then 0#3 else 1#3)
```

Notice that the probability of e_1 evaluating to some value in $(-\infty, 2.5]$ indeed *coincides* with the probability of the discretization of e_1 evaluating to $0_{\#3}$. For e_2, on the other hand, the probability of it evaluating to 2.5 (resp. 3.5) *coincides* with the probability of its discretization evaluating to $0_{\#3}$ (resp. $1_{\#3}$). For this reason, we obtain a semantically equivalent discretization of $e_1 \leq e_2$.

5.0.1 Discretization of Types. We conclude by extending $\mathcal{D}[\![\cdot]\!]$ to types. Figure 14 shows how type τ is discretized to type $\mathcal{D}[\![\tau]\!]$. We then obtain the following natural result, proved in §E.1:

Theorem 5.4 (Discretization Is Well-Typed). *Let* $\Gamma \vdash e : \tau$. *We have* $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]$, *where* $\mathcal{D}[\![\Gamma]\!]$ *transforms all types in* Γ, *i.e.,* $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\Gamma(x)]\!]$ *for all* $x \in \mathrm{dom}(\Gamma)$.

6 Operational Semantics and Soundness

We now establish the soundness of our approach in the following sense: For *every* expression e : **bool**, possibly including *continuous sampling, higher-order functions, recursion, and conditioning*, The intended soundness statement concerns the *uniform fragment*: the syntax in Figure 5 without sequencing, non-strict comparisons,

finite comparisons in source programs, or continuous samplers other than **uniform**. Write $Expr_\tau^{\mathsf{U}}$ for its well-typed expressions and $Expr_\tau^{\mathsf{U}+}$ for the generated-target operational closure. The program-level statement is restricted to closed $e : \mathbf{bool} \in Expr_{\mathbf{bool}}^{\mathsf{U}}$:

$$e : \mathbf{bool} \textit{ and } \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool} \textit{ (obtained from Figure 13) are semantically equivalent.}$$

The first step is to make formal sense of semantic equivalence. For that, we will define a monadic operational semantics for our language in §6.1. We will then proceed by proving the above claim. This is non-trivial: discretization decisions are driven by non-local, type-propagated information, so we cannot proceed by a local rule-by-rule argument. To prove the claim rigorously, we relate the executions of e and $\mathcal{D}[\![e]\!]$ through a coupling-style logical relation in §6.1. The ordinary terminal-limit construction below is rigorous, but it does not by itself prove the displayed statement. A complete proof additionally requires the measurable weak-simulation obligations stated at the end of this section.

6.1 Operational Semantics

The type-directedness of our discretization function $\mathcal{D}[\![\cdot]\!]$ requires our soundness argument to depend on the types of the given (sub-)expressions as well. We thus start by defining a *type-dependent semantic functional*, i.e., we define for every type τ a function $\langle\cdot\rangle : Expr_\tau \to \mathcal{D}(Vals_\tau + \bot)$, mapping each *fully type-annotated* expression $e : \tau$ to a *sub*-probability measure $\langle e : \tau\rangle$ over values of type τ or $\bot$. In the presence of possibly unbounded recursion *and* conditioning, our semantics has to distinguish between non-termination and observation failure, which we realize as follows: The "missing" probability mass in $\langle e : \tau\rangle$ is the probability that e diverges. The probability of $\bot$ is the probability of encountering an observation failure. With this model, semantic equivalence indeed captures our sought-after correctness claim: If $\langle e : \mathbf{bool}\rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle$, then both expressions have the same probability of producing true, false, or an observation failure.

6.1.1 Small Step Semantics. The type-dependent semantic functional $\langle\cdot\rangle$ is defined via a *small-step* semantic functional $[\![\cdot]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$. Defining the latter requires us to carefully design a suitable measurable space over the set of type-annotated expressions of type τ and $\bot$. All details on these measure-theoretic constructions are presented in §F.2. Let us consider the key idea here.

Let $\tau = \mathbf{bool}$ for the sake of concreteness. Care must be taken since, e.g., sub-expressions of type $\mathbf{float}[B; \{3.5, 4.5\}]$ cannot evaluate to *arbitrary* reals but are restricted to the values $\{3.5, 4.5\}$. We tackle this by extracting from each type-annotated expression a suitable *skeleton*, where float constants are replaced by "holes" (hence the term "skeleton"). For instance, the expression

$$e \quad = \quad (x : \mathbf{float}[\{<3.5, <4.5\}; \top] \;<\; y : \mathbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \mathbf{bool}$$

gives rise to the skeleton

$$s \quad = \quad (\square : \mathbf{float}[\{<3.5, <4.5\}; \top] \;<\; \square : \mathbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \mathbf{bool}$$

and the type annotations tell us that the possible values these holes can be filled with are in $\mathbb{R} \times \{3.5, 4.5\}$. We can thus inherit a measurable space over all well-typed expressions arising from this particular skeleton s from the standard Borel measurable space over $\mathbb{R} \times \{3.5, 4.5\}$. The measurable space over *all* well-typed expressions of type $\mathbf{bool}$ (i.e., over $Expr_{\mathbf{bool}}$) is then obtained from the measurable spaces of *all* skeletons that give rise to expressions of type $\mathbf{bool}$.

Equipped with this family of measurable spaces, we define the small-step functional $[\![\cdot]\!]$ in the expected, type-preserving manner (cf. Tables 1 and 2). For instance, if e_1 is not a value, then[1]

$$[\![(\mathbf{let}\ x = e_1 : \tau_1\ \mathbf{in}\ e_2 : \tau_2) : \tau_2]\!] \;=\; \underbrace{[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{(\mathbf{let}\ x = g\ \mathbf{in}\ e_2 : \tau_2) : \tau_2}}_{\text{strict sub-Markov bind with an ordinary-input continuation}} ,$$

i.e., $[\![e_1 : \tau_1]\!]$ yields the sub-distribution over expressions obtained from executing e_1 for *one step*, and the monadic bind $\gg\!=$ "folds" this distribution into the surrounding **let**-statement. The intuition for the remaining statements is analogous. Crucially, $\gg\!=$ propagates the probability of encountering $\bot$ — an observation failure — through executions. For instance, assume that e_1 from above is given by $\mathbf{observe}\ (\text{false} : \mathbf{bool}) : \mathbf{unit}$. Since e_1 encounters an observation failure with probability 1, i.e., $[\![\mathbf{observe}\ (\text{false} : \mathbf{bool}) : \mathbf{unit}]\!](\{\bot\}) = 1$, the

[1] Put formally, $(\mu \gg\!= f)(A) = \int f(x)(A) \cdot \mu(dx)$, see Definition F.28 for details. δ_e denotes the Dirac distribution of e.

same holds for the surrounding **let**-statement, i.e., also $[\![(\textbf{let}\ x = e_1 : \tau_1\ \textbf{in}\ e_2 : \tau_2) : \tau_2]\!](\{\bot\}) = 1$. All details — including proofs that all monadic binds are well-behaved since they operate on suitable Markov kernels — are provided in §F.4The appendix defines this strict bind by extending its ordinary-input continuation canonically at $\bot$. The remaining measurability obligations for the small-step kernel are stated explicitly there.

6.1.2 From Small-Step to Big-Step Semantics. We now define $\langle\cdot\rangle$ via $[\![\cdot]\!]$. For that, we first lift the one-step semantics $[\![\cdot]\!]$ to an n-step semantics $\langle\cdot\rangle_n$ in the expected way:

$$\langle e : \tau\rangle_0 = \delta_{e:\tau} \qquad \text{and} \qquad \langle e : \tau\rangle_{n+1} \;=\; \langle e : \tau\rangle_n \gg\!= \lambda e'.\, \begin{cases} \delta_\bot & \text{if } e' = \bot \\ [\![e' : \tau]\!] & \text{otherwise} \end{cases}$$

This construction is well-defined since the functional $[\![\cdot]\!]$ itself gives rise to a suitable Markov sub-Markov kernel (Theorem F.39). We then define for every type τ the functional $\langle\cdot\rangle\colon Expr_\tau \to \mathcal{D}(Vals_\tau + \bot)$ as $\langle e : \tau\rangle = \lambda A.\, \lim_{n\to\infty}\langle e : \tau\rangle_n(A)$, which is well-defined by the monotoneconvergence theorem: the sequence $(\langle e : \tau\rangle_n)_{n\in\mathbb{N}}$ is monotonically increasing The full measures $\langle e : \tau\rangle_n$ on expressions are not monotone: a step moves mass away from the current nonterminal expression. Instead, let ν_n^e be their restrictions to the measurable terminal subspace $Vals_\tau + \bot$. Terminal outcomes are absorbing, so for every measurable set A over $Vals_\tau + \bot$ (cf. Lemma F.41)and upper-bounded by 1: executing a program for more and more steps can only increase the probability of terminating in a given set of values or an observation failure$A \subseteq Vals_\tau + \bot$,

$$\nu_n^e(A) \le \nu_{n+1}^e(A).$$

We define

$$\langle e : \tau\rangle(A) = \lim_{n\to\infty} \nu_n^e(A).$$

The appendix proves the displayed eventwise monotonicity and uses monotone convergence to establish countable additivity of the limit (Lemmas F.41 and F.43). Thus the limit is a subprobability measure, not merely a pointwise limit of set functions.

6.2 Soundness via Probabilistic Couplings

We first state the central theorem , then outline our coupling-style correctness argument. Let $e : \textbf{bool}$. Then $\langle e : \textbf{bool}\rangle = \langle \mathcal{D}[\![e : \textbf{bool}]\!] : \textbf{bool}\rangle$The desired contextual-equivalence theorem is not established by the current synchronous tables. A proof requires a measurable weak simulation closed under generated-target administrative steps, an error-preserving projection defined at $\bot$, and a proof that the weak target iteration has the same terminal limit as the ordinary target semantics. Intuitively, we prove that the two programs can execute in a synchronized manner such that their probabilities of terminating in value v coincide for each $v \in \{\text{true}, \text{false}, \bot\}$. The exact graph of the transformation does not establish these obligations, so this section claims neither a coupled finite-step projection nor contextual equivalence.

Towards this end, define for every type τ the set $P_\tau = \{(e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]) \mid e : \tau\}$. We define a *coupled small-step semantics* $[\![\cdot,\cdot]\!]\colon P_\tau \to \mathcal{D}(P_\tau + \bot)$ (cf. ????), which models the simultaneous execution of e and $\mathcal{D}[\![e : \tau]\!]$. The key property of this coupled semantics is that the semantics of both e and $\mathcal{D}[\![e : \tau]\!]$ can be recovered in a sense we formalize next. The sub-Markov kernel definition and its closure under error extension and composition are supplied by the separate kernel-category repair. This branch uses those results for the ordinary limit construction; it does not duplicate them here.

Given $\mu \in \mathcal{D}(P_\tau + \bot)$, we define the *projection of μ onto the i-th component* for $i \in \{1, 2\}$ as With projections, we recover the semantics of e and $\mathcal{D}[\![e : \tau]\!]$ as follows: Let $e : \tau$. We have:

$$\pi_1([\![e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![e : \tau]\!].$$

$$\pi_2([\![e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![\mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!].$$

Now, using a limit construction analogous to §6.1.2 for obtaining a coupled *big-step* semantics $\langle\cdot,\cdot\rangle : P_\tau \to \mathcal{D}(V_\tau + \bot)$, where $V_\tau = \{(v : \tau, \mathcal{D}[\![v]\!] : \mathcal{D}[\![\tau]\!]) \mid v : \tau \text{ is a value}\}$, we prove a suitable continuity property of projection (cf. **??**) to conclude that **??** transfers to $\langle\cdot,\cdot\rangle$. Finally, observing that $V_{\texttt{bool}} + \bot = \{(\text{true}, \text{true}), (\text{false}, \text{false}), \bot\}$ yields the desired claim: ■

7 Implementation, Case Studies, and Evaluation

We have implemented the Slice type inference and discretization procedures in OCaml. The implementation is open-source and available at https://github.com/[anonymized], together with all benchmarks, scripts, and a Docker image for reproduction. To compute the interval masses of continuous distributions, we use the OCaml GSL package [19] for cumulative distribution function calculations. All experiments were run in this Docker image on an Apple M3 with 16 GB of RAM.

This section is driven by two research questions:

(1) Does Slice enable exact inference on programs combining continuous sampling, higher-order functions, and unbounded recursion?
(2) On programs that are within the scope of existing tools for exact inference, is the combination of Slice and discrete inference back-ends competitive?

For the first question, we provide a collection of case studies that involve continuous distributions together with recursion, higher-order functions, lists, or conditioning in §7.1. For these case studies, we use Slice to transform the original programs to finite discrete models and analyze the resulting program with Storm [16] — a probabilistic model checker.

For the second question, we evaluate Slice+Dice and Slice+Roulette on two classes of benchmarks in Sections 7.2 and 7.3: (i) synthetic scaling benchmarks that let us vary program size and dependency structure in a controlled way, and (ii) benchmark suites from prior work [22, 40]. We compare against Sppl [40], an exact inference engine supporting continuous probabilistic programs, on benchmarks that *both* Slice and Sppl support, and we additionally report Psi and Hakaru timings for the representative prior-work benchmarks. We remark, however, that Sppl supports arithmetic, as well as (soft) conditioning on measure zero events, which Slice does not support. Our evaluation merely serves the purpose of investigating (i) the overhead of discretization and (ii) the benefits of being able to use efficient *BDD-based* inference back-ends (i.e., Dice and Roulette) for exact inference on *continuous programs*.

7.1 Slice+Storm: Case Studies

There is no exact inference system that, to our knowledge, can cover programs combining all three features: continuous sampling, higher-order functions, and unbounded recursion. As a consequence, there is no exact inference engine for the programs in Sections 2.6, 7.1.2 and 7.1.3. Instead, we use *probabilistic model checking*: The small-step semantics of our language (cf. Tables 1 and 2) for discrete programs gives rise to a *Markov chain* modeling the stochastic process of program execution, inspired by [2] on imperative programs. If that Markov chain is finite, we can use the probabilistic model checker Storm [16] for exact inference. Let us start with an illustrative example.

7.1.1 Conditional Resampling Loop. Consider the continuous program depicted in Figure 15, containing continuous sampling, unbounded recursion, and conditioning inside of the recursion. Slice automatically discretizes this program, also shown in Figure 15. We generate the corresponding Markov chain by unfolding the small-step semantics in a breadth-first manner. In the presence of unbounded recursion, this Markov chain *might* be infinite and this process might thus diverge. If, however, it terminates, the resulting Markov chain is finite and precisely captures the execution behavior of the discretized program. For this example, this is the case: Even though the discretized program contains unbounded recursion, its execution behavior can be modelled as a finite-state Markov chain[2], where $\bot$ is a special state entered when encountered an observation failure.

Now, using the probabilistic model checker Storm, we can perform exact inference: It is immediate that the Markov chain's probability $\Pr(\Diamond\texttt{true} \mid \Box\neg\bot)$ of eventually reaching true (in LTL: $\Diamond$true) *given* that we never

[2]We apply a straightforward optimization: Intermediate execution steps that do not branch probabilistically can be collapsed into a single transition. This is the reason why all states in Figure 15 correspond to the header of the recursive function.

(iii) Closure under countable unions: Let $(C_n)_{n\in\mathbb{N}}$ be a sequence with $C_n \in \mathcal{F}_{Expr_\tau+\bot}$ for all n. For each n, define $A_n := C_n \cap Expr_\tau \in \mathcal{F}_{Expr_\tau}$, where the latter holds since C_n is of the form A or $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$, and in either case $C_n \cap Expr_\tau = A$.

Let $A := \bigcup_{n\in\mathbb{N}} A_n$. Since $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $A \in \mathcal{F}_{Expr_\tau}$. Now observe:

$$\bigcup_{n\in\mathbb{N}} C_n = A \cup \left(\bigcup_{n\in\mathbb{N}} (C_n \setminus Expr_\tau)\right).$$

The second factor is either $\emptyset$ (if every $C_n = A_n$) or $\{\bot\}$ (if some $C_n = A_n \cup \{\bot\}$). In the first sub-case, $\bigcup_n C_n = A$ is of the form A for some $A \in \mathcal{F}_{Expr_\tau}$; in the second, $\bigcup_n C_n = A \cup \{\bot\}$ is of the form $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$. In both cases $\bigcup_n C_n \in \mathcal{F}_{Expr_\tau+\bot}$. □

Finally, we can equip this measurable space $(Expr_\tau + \bot, \mathcal{F}_{Expr_\tau+\bot})$ with a subprobability measure such that $\mu(Expr_\tau + \bot) \le 1$.

We give some basic lemmas:

Lemma F.27. *Let $(Expr_\tau + \bot, \mathcal{F}_{Expr_\tau+\bot})$ be the Borel measurable space of well-typed expressions with error from Definition F.25. Let $Vals_\tau \subset Expr_\tau$ be the set of all well-typed value expressions. Let $N_\tau \subset Expr_\tau$, defined as $N = Expr_\tau \setminus Vals_\tau$, be the set of all well-typed non-value expressions. Let $\{\bot\}$ be the singleton set containing the error element $\bot$. The following are true:*

(1) The set $Vals_\tau \in \mathcal{F}_{Expr_\tau+\bot}$.
(2) The set $\{\bot\} \in \mathcal{F}_{Expr_\tau+\bot}$.
(3) The set $N_\tau \in \mathcal{F}_{Expr_\tau+\bot}$.

Proof. We prove the measurability of $Vals_\tau$, $\{\bot\}$, and N_τ in separate parts. For each, we show that it belongs to the σ-algebra $\mathcal{F}_{Expr_\tau+\bot}$ on the extended expression space $Expr_\tau + \bot$.

(1) **(Measurability of $Vals_\tau$):** Every expression skeleton $s \in Skel_\tau$ is either a value skeleton or a non-value skeleton, depending on whether the syntactic form of s is a value. An expression $e \in Expr_\tau$ is a value iff its skeleton is a value skeleton.
To show $Vals_\tau \in \mathcal{F}_{Expr_\tau}$, by Definition F.23, it suffices to show that $\forall s \in Skel_\tau, Vals_\tau \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$.
Fix $s \in Skel_\tau$:
- If s is a value skeleton, then every expression in $Expr_{s;\tau}$ is a value, so $Vals_\tau \cap Expr_{s;\tau} = Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$.
- If s is a non-value skeleton, then no expression in $Expr_{s;\tau}$ is a value, so $Vals_\tau \cap Expr_{s;\tau} = \emptyset \in \mathcal{F}_{Expr_{s;\tau}}$.

Thus, $Vals_\tau \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$ for any $s \in Skel_\tau$, so $Vals_\tau \in \mathcal{F}_{Expr_\tau} \subseteq \mathcal{F}_{Expr_\tau+\bot}$.

(2) **(Measurability of $\{\bot\}$):** Taking $A = \emptyset$ in Definition F.25 gives $\{\bot\} \in \mathcal{F}_{Expr_\tau+\bot}$.

(3) **(Measurability of N_τ):** $N_\tau = Expr_\tau \setminus Vals_\tau$ is the complement of $Vals_\tau$ in $Expr_\tau$. Since $Vals_\tau \in \mathcal{F}_{Expr_\tau}$ and $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $N_\tau \in \mathcal{F}_{Expr_\tau} \subseteq \mathcal{F}_{Expr_\tau+\bot}$.

□

Note that in general, given (X, Σ_X) and (Y, Σ_Y) to be measurable spaces, $((X \times Y) + \bot)$ forms a measurable space, since the product $(X \times Y)$ is measurable and for any measurable space (Z, Σ_Z), the space $(Z + \bot, \Sigma_{Z+\bot})$ with $\Sigma_{Z+\bot}$ generated by $\Sigma_Z \cup \bot$ is measurable.

F.3 Distribution Monad Strict Sub-Markov Kleisli Composition with Error

Let X be a set equipped with a suitable measurable space so that $\mathcal{D}(X + \bot)$ denotes the set of probability measures over $X + \bot$. We will make heavy use of the following generic monad construction, for which one easily verifies the monad laws: measurable space and let $\mathcal{D}(X + \bot)$ denote the subprobability measures on its error extension.

Definition F.28 (Strict sub-Markov Kleisli bind). We define

$$\begin{aligned}&\text{return}\colon X \to \mathcal{D}(X+\bot), \quad \text{return}(x) = \delta_x\\ &\text{bind}\colon \mathcal{D}(X+\bot) \to (X \to \mathcal{D}(Y+\bot)) \to \mathcal{D}(Y+\bot),\\ &\text{bind}(\mu)(f)(A) = \int_{X+\bot} f_\bot(x)(A)\cdot \mu(dx)\,,\end{aligned}$$

where δ_x denotes the Dirac measure for x, and f is a Markov kernel . $f : X \to \mathcal{D}(Y+\bot)$ is a sub-Markov kernel and $f_\bot$ is its canonical extension from Definition F.29. Thus the continuation is specified only on ordinary inputs and $\delta_\bot \gg\!= f = \delta_\bot$; bind cannot recover from an error.

Definition F.29 ($\bot$-extension). Define the $\bot$-extension of $g : X \to \mathcal{D}(Y+\bot)$ to be $g_\bot\colon (X+\bot) \to \mathcal{D}(Y+\bot)$, obtained from g by:

$$g_\bot(e) = \begin{cases}\delta_\bot & \text{if } e = \bot\\ g(e) & \text{otherwise}\,.\end{cases}$$

Lemma F.30 ($\bot$-extension is a sub-Markov kernel). *Given a Markov sub-Markov kernel $f : X \to \mathcal{D}(Y+\bot)$, its $\bot$ extension $f_\bot : (X+\bot) \to \mathcal{D}(Y+\bot)$ is also a Markov kernel. sub-Markov kernel. If f is Markov, its extension is Markov.*

Proof. We verify the two conditions for $f_\bot$ to be a Markov sub-Markov kernel.

(1) Probability Subprobability measure: We show for each $e \in X+\bot$, the map

$$A \mapsto f_\bot(e)(A)$$

is a probability measure.

Fix $e \in X+\bot$.

- If $e = \bot$, then $f_\bot(\bot) = \delta_\bot$, the Dirac measure at $\bot$, which is a probability measure on $Y+\bot$.
- If $e \in X$, then $f_\bot(e) = f(e)$, and since f is a Markov kernel by assumption, $f(e)$ is a probability measure on $Y+\bot$a subprobability measure by assumption.

Thus , for all $e \in X+\bot$, the map $A \mapsto f_\bot(e)(A)$ is a probability each map $A \mapsto f_\bot(e)(A)$ is a subprobability measure.

(2) Measurable function: We show for each $A \in \mathcal{F}_{Y+\bot}$, the map $e \mapsto f_\bot(e)(A)$ is a measurable function.

Fix $A \in \mathcal{F}_{Y+\bot}$. Define

$$g\colon (X+\bot) \to [0,1], \qquad g(e) := f_\bot(e)(A).$$

We prove g is measurable by showing $g^{-1}(B) \in \mathcal{F}_{X+\bot}$ for every $B \in \mathcal{B}([0,1])$. It suffices to prove that for every $a \in \mathbb{R}$,

$$g^{-1}((a,\infty)\cap[0,1]) \in \mathcal{F}_{X+\bot}$$

because the family of sets $\{(a,\infty)\cap[0,1] : a \in \mathbb{R}\}$ generates $\mathcal{B}([0,1])$.

Because $g(e) \in [0,1]$ for all $e \in X+\bot$ by definition, we have for every $a \in \mathbb{R}$,

$$\begin{aligned}g^{-1}\big((a,\infty)\cap[0,1]\big) &= \{e \in X+\bot \mid g(e) \in (a,\infty)\cap[0,1]\}\\ &= \{e \in X+\bot \mid (g(e)\in(a,\infty)) \wedge (g(e)\in[0,1])\}\\ &= \{e \in X+\bot \mid (g(e) > a) \wedge (g(e)\in[0,1])\}\\ &= \{e \in X+\bot \mid g(e) > a\},\end{aligned}$$

where the last step uses that $g(e) \in [0,1]$ always holds. Hence it suffices to show that $\{e \in X+\bot \mid g(e) > a\} \in \mathcal{F}_{X+\bot}$ for every $a \in \mathbb{R}$.

- If $a \geq 1$: $\{e \in X+\bot \mid g(e) > a\} = \emptyset \in \mathcal{F}_{X+\bot}$.
- If $a < 0$: $\{e \in X+\bot \mid g(e) > a\} = X+\bot \in \mathcal{F}_{X+\bot}$.
- If $0 \leq a < 1$: $\{e \in X+\bot \mid g(e) > a\}$.

The only nontrivial case is the last. It splits into two sets:

$$\begin{aligned}\{e \in X + \bot \mid g(e) > a\} &= \{e \in X + \bot \mid f_\bot(e)(A) > a\} \\ &= \{e \in X \mid f_\bot(e)(A) > a\} \cup \{\bot \mid f_\bot(e)(A) > a\} \\ &= \{e \in X \mid f(e)(A) > a\} \cup \{\bot \mid \delta_\bot(A) > a\}\end{aligned}$$

The first set is measurable in X (and hence $X+\bot$) since f is a Markov kernelby assumptionsub-Markov kernel. The second set is either $\emptyset \in \mathcal{F}_{X+\bot}$ or $\{\bot\} \in \mathcal{F}_{X+\bot}$. Since $\mathcal{F}_{X+\bot}$ is closed under countable unions, then $g^{-1}(B) \in \mathcal{F}_{X+\bot}$ for every $B \in \mathcal{B}([0,1])$. Thus, for all $A \in \mathcal{F}_{Y+\bot}$, the map $e \mapsto f_\bot(e)(A)$ is a measurablefunction$\emptyset$ or $\{\bot\}$, so the union is measurable.

Since both conditions hold, $f_\bot$ is a Markov kernel. sub-Markov kernel. If f is Markov, both cases have total mass one, so $f_\bot$ is Markov. □

Lemma F.31 (Dirac return is a Markov kernel). *For any measurable space $(X, \mathcal{F}_X)$, the map*

$$\mathsf{return} : X \to \mathcal{D}(X + \bot), \qquad x \mapsto \delta_x.$$

is a Markov kernel.

Proof. We verify the two conditions for return to be a Markov kernel.

(1) Probability measure: We show for each $x \in X$, the map

$$A \mapsto \mathsf{return}(x)(A) = \delta_x(A)$$

is a probability measure.

Fix $x \in X$. δ_x is the Dirac measure at x, hence a probability measure on $X + \bot$. Therefore, for every $x \in X$, the map $A \mapsto \delta_x(A)$ is a probability measure.

(2) Measurable function: We show that for each $A \in \mathcal{F}_{X+\bot}$, the map

$$x \mapsto \mathsf{return}(x)(A) = \delta_x(A)$$

is a measurable function.

Fix $A \in \mathcal{F}_{X+\bot}$. Define

$$g : X \to [0,1], \qquad g(x) := \delta_x(A).$$

We prove g is measurable by showing $g^{-1}(B) \in \mathcal{F}_X$ for every $B \in \mathcal{B}([0,1])$. It suffices to prove that for every $a \in \mathbb{R}$,

$$g^{-1}\big((a,\infty) \cap [0,1]\big) \in \mathcal{F}_X,$$

because the family $\{(a,\infty) \cap [0,1] : a \in \mathbb{R}\}$ generates $\mathcal{B}([0,1])$.

Because $g(e) \in [0,1]$ for all $e \in X + \bot$, we have for every $a \in \mathbb{R}$,

$$\begin{aligned}g^{-1}\big((a,\infty) \cap [0,1]\big) &= \{e \in X \mid g(e) \in (a,\infty) \cap [0,1]\} \\ &= \{e \in X \mid (g(e) \in (a,\infty)) \wedge (g(e) \in [0,1])\} \\ &= \{e \in X \mid (g(e) > a) \wedge (g(e) \in [0,1])\} \\ &= \{e \in X \mid g(e) > a\},\end{aligned}$$

where the last step uses that $g(e) \in [0,1]$ always holds. Hence it suffices to show that $\{e \in X \mid g(e) > a\} \in \mathcal{F}_X$ for every $a \in \mathbb{R}$.

- If $a \geq 1$: $\{x \in X \mid g(x) > a\} = \emptyset \in \mathcal{F}_X$.
- If $a < 0$: $\{x \in X \mid g(x) > a\} = X \in \mathcal{F}_X$.
- If $0 \leq a < 1$: $\{x \in X \mid g(x) > a\}$.

The only nontrivial case is the last. Using $g(x) = \delta_x(A)$ and the fact that $\delta_x(A) \in \{0, 1\}$ for all $x \in X$, we have for $0 \leq a < 1$:

$$\begin{aligned}\{x \in X \mid g(x) > a\} &= \{x \in X \mid \delta_x(A) > a\} \\ &= \{x \in X \mid \delta_x(A) = 1\} \\ &= \{x \in X \mid x \in A\} \\ &= A \cap X.\end{aligned}$$

Since $A \in \mathcal{F}_{X+\perp}$ and $\mathcal{F}_X = \{X \cap A \mid A \in \mathcal{F}_{X+\perp}\}$ is the subspace σ-algebra on $X \subseteq X + \perp$, it follows that $A \cap X \in \mathcal{F}_X$.

Therefore $\{x \in X \mid g(x) > a\} \in \mathcal{F}_X$ for all $a \in \mathbb{R}$, so $g^{-1}\big((a, \infty) \cap [0, 1]\big) \in \mathcal{F}_X$ for all a. Hence $g^{-1}(B) \in \mathcal{F}_X$ for every $B \in \mathcal{B}([0, 1])$, and g is measurable. □

Lemma F.32 (Zero kernel is a sub-Markov kernel). *Let $(X, \mathcal{F}_X)$ and $(Y, \mathcal{F}_Y)$ be measurable spaces. The constant-zero map*

$$f : X \to \mathcal{D}(Y), \qquad f(x)(A) := 0$$

is a Markov sub-Markov kernel.

Proof. We verify the two kernel conditions.

(1) Probability Subprobability measure. We show for each $x \in X$, the map

$$A \mapsto f(x)(A)$$

is a probability measure.

Fix any $x \in X$. Then $f(x)$ is the zero measure: it satisfies σ-additivity since

$$f(x)\left(\bigcup_i A_i\right) = 0 = \sum_i 0 = \sum_i f(x)(A_i)$$

for every pairwise disjoint family $(A_i)_i$, and total mass $f(x)(Y) = 0 \leq 1$.

(2) Measurable function. For any $A \in \mathcal{F}_Y$, the map $x \mapsto f(x)(A) = 0$ is the constant-zero function on X, hence measurable.

Thus both conditions hold, so f is a Markov sub-Markov kernel. □

Lemma F.33 (Composition of sub-Markov kernels). *Let (X, Σ_X), (Y, Σ_Y), and (Z, Σ_Z) be measurable spaces. If*

$$P : X \to \mathcal{D}(Y) \quad \text{and} \quad Q : Y \to \mathcal{D}(Z)$$

are Markov sub-Markov kernels, then their composition

$$(P \gg\!= Q)(A) \;:=\; \int_{x \in supp(P)} Q(x)(A)\, P(dx)$$

defines a Markov sub-Markov kernel

$$P \gg\!= Q : X \to \mathcal{D}(Z).$$

Proof. See [30, Lemma 1.38]. □

We claim proving $C \in \mathcal{F}_{Expr_{\tau_1}}$ is sufficient because by Definition F.1, this gives $\mathrm{Sub}_{e,x}^{-1}(A) = Val_{\tau_1} \cap C \in \mathcal{F}_{Val_{\tau_1}}$. To show $C \in \mathcal{F}_{Expr_{\tau_1}}$, by Definition F.23, it suffices to show that $\forall s \in Skel_{\tau_1}, C \cap Expr_{s;\tau_1} \in \mathcal{F}_{Expr_{s;\tau_1}}$.

Fix $s \in Skel_{\tau_1}$. We must prove:

$$\begin{aligned}
& C \cap Expr_{s;\tau_1} \in \mathcal{F}_{Expr_{s;\tau_1}} \\
\iff & \{u \in Expr_{\tau_1} \mid e[u/x] \in A\} \cap Expr_{s;\tau_1} \in \mathcal{F}_{Expr_{s;\tau_1}} && \text{(definition of } C\text{)} \\
\iff & \{u \in Expr_{s;\tau_1} \mid e[u/x] \in A\} \in \mathcal{F}_{Expr_{s;\tau_1}} && (Expr_{s;\tau_1} \subseteq Expr_{\tau_1}) \\
\iff & \{s[\vec{r}] \in Expr_{s;\tau_1} \mid e[s[\vec{r}]/x] \in A\} \in \mathcal{F}_{Expr_{s;\tau_1}} && \text{(rewrite } u\text{)} \\
\iff & \{\vec{r} \in A_{s;\tau_1} \mid e[s[\vec{r}]/x] \in A\} \in \mathcal{F}_{s;\tau_1} && \text{(Lemma F.21)}
\end{aligned}$$

Define

$$R_s := \{\vec{r} \in A_{s;\tau_1} \mid e[s[\vec{r}]/x] \in A\}.$$

Let $t \in Skel_{\tau_2}$ be the skeleton obtained from e by replacing each occurrence of x with s. Define a constant extension map:

$$\psi_s : A_{s;\tau_1} \to A_{t;\tau_2}, \qquad \psi_s(\vec{r}) = \phi(\vec{r}, \vec{c_1}, ..., \vec{c_k})$$

where $\vec{c_1}, ..., \vec{c_k} \in \mathbb{R}$ are the float constants appearing in e (other than at occurrences of x), and ϕ permutes/interleaves the blocks into the order dictated by a left-to-right traversal of the holes of t.

More precisely, since t is obtained from e by replacing each free occurrence of x with s, the holes of t consist exactly of:

- the holes of s, repeated once per occurrence of x in e, and
- the holes coming from the float constants of e itself.

By Lemma F.35, ψ_s is measurable.

By construction, for all $\vec{r} \in A_{s;\tau_1}$:

$$e[s[\vec{r}]/x] = t[\psi_s(\vec{r})].$$

Define the measurable set of hole-assignments corresponding to A at skeleton t:

$$\begin{aligned}
A_t :=& \{\vec{u} \in A_{t;\tau_2} \mid t[\vec{u}] \in A\} \\
:=& \{\vec{u} \in A_{t;\tau_2} \mid t[\vec{u}] \in A \cap Expr_{t;\tau_2}\} && (t[\vec{u}] \text{ ranges over } Expr_{t;\tau_2})
\end{aligned}$$

By Definition F.23, $A \cap Expr_{t;\tau_2} \in \mathcal{F}_{Expr_{t;\tau_2}}$, and by Lemma F.3, $A_t \in \mathcal{F}_{t;\tau_2}$. We compute:

$$\begin{aligned}
R_s &= \{\vec{r} \in A_{s;\tau_1} \mid e[s[\vec{r}]/x] \in A\} \\
&= \{\vec{r} \in A_{s;\tau_1} \mid t[\psi_s(\vec{r})] \in A\} \\
&= \{\vec{r} \in A_{s;\tau_1} \mid \psi_s(\vec{r}) \in A_t\} \\
&= \psi_s^{-1}(A_t).
\end{aligned}$$

Since ψ_s is measurable and $A_t \in \mathcal{F}_{t;\tau_2}$, we have $R_s = \psi_s^{-1}(A_t) \in \mathcal{F}_{s;\tau_1}$.

Since $R_s \in \mathcal{F}_{s;\tau_1}$ for every $s \in Skel_{\tau_1}$, we get $C \in \mathcal{F}_{Expr_{\tau_1}}$, and therefore

$$\mathrm{Sub}_{e,x}^{-1}(A) = C \cap Val_{\tau_1} \in \mathcal{F}_{Val_{\tau_1}}.$$

Hence $\mathrm{Sub}_{e,x}$ is measurable. □

Conjecture F.38 (Strict bind continuations). *Every ordinary-input continuation used by the small-step rules is a sub-Markov kernel on the stated expression spaces. This requires the substitution and primitive-operation measurability obligations developed above; the incomplete case analysis below is not used as a proof.*

Conjecture F.39 (Small-step semantics is a sub-Markov kernel). *For Conditional on the sampler and case-partition measurability premises, for each type τ , the small-step operational semantics $[\![\cdot]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ defines a Markov kernel. $[\![\cdot]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ is a sub-Markov kernel.*

F.5 Small-Step Coupling Semantics

F.6 Big-Step Semantics and Correctness of Discretization

We extend both of the small-step semantics canonically to an This construction uses the strict sub-Markov bind defined in the kernel-category development: bind accepts an ordinary continuation and itself extends it by sending $\bot$ to $\delta_\bot$. Let $S_\tau : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ be the ordinary one-step kernel, $S_\tau(e) = [\![e : \tau]\!]$. Define the ordinary n-step semantics byfor every typed expression state by

$$\langle e : \tau \rangle_n = \begin{cases} \delta_{e:\tau} & \text{if } n = 0, \\ \langle e : \tau \rangle_{n-1} \gg\!= S_\tau & \text{if } n > 0. \end{cases}$$

As a sanity check, we show that the n-step semantics defines a Markov sub-Markov kernel.

Lemma F.40 (n-step semantics is a sub-Markov kernel). *For all $n \in \mathbb{N}$, $\langle \cdot \rangle_n : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ is a Markov sub-Markov kernel.*

Proof. By induction on n.

Base case: $n = 0$. By definition,

$$\langle e : \tau \rangle_n = \delta_{e:\tau}$$

i.e. the map $e \mapsto \delta_e$. By Lemma F.31, this is a Markov kernel.

Inductive step: $n > 0$. By definition,

$$\langle e : \tau \rangle_n = \langle e : \tau \rangle_{n-1} \gg\!= S_\tau$$

By the I.H., $\langle \cdot \rangle_{n-1} : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ is a Markov kernel. By Theorem F.39, $\lambda e'.[\![e' : \tau]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ is a Markov kernel. By Lemma F.30, its $\langle \cdot \rangle_{n-1}$ is a sub-Markov kernel. The ordinary one-step semantics is sub-Markov, and strict sub-Markov bind extends it at $\bot$ -extension from $(Expr_\tau + \bot)$ to $\mathcal{D}(Expr_\tau + \bot)$ is also a Markov kernel. By Lemma F.33, $\langle e : \tau \rangle_{n-1} \gg\!= \lambda e'.[\![e' : \tau]\!]$ is a Markov and composes the kernels. Hence $\langle e : \tau \rangle_{n-1} \gg\!= S_\tau$ is a sub-Markov kernel. □

For every type τ, we denote by $Vals_\tau$ the set of values of type τ and equip it with a suitable measurable sub-space of the measurable space over $Expr_\tau + \bot$ (cf. Lemma F.27.1). Moreover, let $V_\tau = \{(v : \tau, \mathcal{D}[\![v]\!] : \mathcal{D}[\![\tau]\!]) \mid v : \tau\}$.

Let $e : \tau \in Expr_\tau$ and $n \in \mathbb{N}$. Let $Vals_\tau$ Let $T_\tau = Vals_\tau + \bot$ be the measurable set of values. We have:

$\langle e : \tau \rangle_n(Vals_\tau) \le \langle e : \tau \rangle_{n+1}(Vals_\tau)$

$\langle e : \tau \rangle_n(\bot) \le \langle e : \tau \rangle_{n+1}(\bot)$

By induction on subspace of terminal outcomes. For $e : \tau$, define the restriction of its n.

Base case: $n = 0$.

We must show $\langle e : \tau \rangle_0(Vals_\tau) \le \langle e : \tau \rangle_1(Vals_\tau)$.

By definition, -step semantics to terminal outcomes by

$$\nu_n^e(A) = \langle e : \tau \rangle_n(A) \qquad (A \in \mathcal{F}_{T_\tau}).$$

Each ν_n^e is a subprobability measure because it is a restriction of an n-step subprobability measure. Notice that the full n-step measures on expressions need not be increasing: a deterministic step removes mass from the singleton containing its source expression.

Therefore,

Lemma F.41 (Increasing terminal restrictions). *For every $e : \tau$, $n \in \mathbb{N}$, and measurable terminal event $A \in \mathcal{F}_{T_\tau}$,*

$$v^e_n(A) \le v^e_{n+1}(A).$$

Now, for $n = 1$, by definition,

Proof. Let $(S_\tau)_\perp$ be the error-total continuation used internally by strict bind. Terminal outcomes are absorbing: $(S_\tau)_\perp(x) = \delta_x$ when $x \in T_\tau$. Consequently, for every $x \in Expr_\tau + \perp$ and measurable $A \subseteq T_\tau$,

$$(S_\tau)_\perp(x)(A) \ge \mathbf{1}_A(x).$$

We analyze the two cases for e:

Case 1: e is a value v. Then, $\langle e : \tau\rangle_0(Vals_\tau) = 1$.

For $n = 1$: For a nonterminal x the right-hand side is zero; for a terminal x the two sides are equal. Hence

$$\begin{aligned} v^e_{n+1}(A) &= \int (S_\tau)_\perp(x)(A)\, \langle e : \tau\rangle_n(dx) \\ &\ge \int \mathbf{1}_A(x)\, \langle e : \tau\rangle_n(dx) = v^e_n(A). \end{aligned}$$

From the semantics, $[\![v : \tau]\!] = \delta_{v:\tau}$, which means $\langle e : \tau\rangle_1 = \delta_{v:\tau}$, and thus and thus $\langle e : \tau\rangle_0(Vals_\tau) \le \langle e : \tau\rangle_1(Vals_\tau)$. □

Case 2: e is an expression. Then, $\langle e : \tau\rangle_0(Vals_\tau) = 0$.

Definition F.42 (Big-step semantics). For $n = 1$: $e : \tau$, define a measure on terminal outcomes pointwise by

$$\langle e : \tau\rangle(A) = \lim_{n\to\infty} v^e_n(A), \qquad A \in \mathcal{F}_{T_\tau}.$$

From the semantics, $[\![e : \tau]\!]$ may step to either a value, which means $[\![e : \tau]\!](Vals_\tau) \ge 0$, or an expression, which means $[\![e : \tau]\!](Vals_\tau) = 0$. In either case, and thus $\langle e : \tau\rangle_0(Vals_\tau) \le \langle e : \tau\rangle_1(Vals_\tau)$.

Inductive step: We want to show that for $n = k + 1$:

By the recursive definition:

By the IH,

We do a case analysis on e' and e'', whether they are in N_τ or $Vals_\tau$.

Case $e' \in Vals_\tau$, $e'' \in Vals_\tau$.

Then, from the semantics, $[\![e']\!]$ and $[\![e'']\!]$ are both Diracs over a value. Combined with the IH and the fact that $[\![\cdot]\!]$ does not change the probability mass of $\cdot$ if it is The semantic kernel is defined on all typed expression states so it can be iterated. Any program-level comparison with the transformed term is restricted to a value,

Case $e' \in Vals_\tau$, $e'' \in N_\tau$closed, well-typed initial expression.

Then, from the semantics, $[\![e']\!]$ is a Dirac over a value and $[\![e'']\!]$ is a distribution over a non-value expression . Combined with the IH and the fact that $[\![\cdot]\!]$ does not change the probability mass of $\cdot$ if it is a value and that $[\![\cdot]\!]$ does not add any probability mass to $Vals_\tau$ if $\cdot$ is a non-value expression,

Case $e' \in Vals_\tau, e'' \in N_\tau$.

Analogous to the previous case, but with e' and e'' switched.

Case $e' \in N_\tau, e'' \in N_\tau$.

Then, from the semantics, $[\![e']\!]$ and $[\![e'']\!]$ are both distributions over a non-value expression. Combined with the IH and the fact that $[\![\cdot]\!]$ does not add any probability mass to $Vals_\tau$ if $\cdot$ is a non-value expression,

In all cases, $\langle e : \tau\rangle_{k+1}(Vals_\tau) \leq \langle e : \tau\rangle_{k+2}(Vals_\tau)$.

Let $(e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]) \in P_\tau$ and $n \in \mathbb{N}$. Let V_τ be the measurable set of value pairs. We have:

$\langle e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]\rangle_n(V_\tau) \leq \langle e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]\rangle_{n+1}(V_\tau)$

$\langle e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]\rangle_n(\bot) \leq \langle e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]\rangle_{n+1}(\bot)$

Lemma F.43 (Big-step semantics is well-defined). *The set function in Definition F.42 is a subprobability measure. Moreover, as a function of its initial expression, it is a sub-Markov kernel into the terminal measurable space.*

Proof. By induction on n. Analogous to proof of Lemma F.41.

Let $e : \tau$. The *(big-step) semantics* $\langle e : \tau\rangle \in \mathcal{D}(Vals_\tau + \bot)$ and $\langle e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]\rangle \in \mathcal{D}(V_\tau + \bot)$ are respectively defined as Lemma F.41, each limit exists and equals the corresponding supremum. We prove countable additivity rather than infer it merely from pointwise convergence. If $(A_j)_{j\in\mathbb{N}}$ are pairwise disjoint measurable subsets of T_τ, then

$$\begin{aligned}\langle e : \tau\rangle\left(\bigcup_j A_j\right) &= \lim_n \sum_j \nu_n^e(A_j)\\ &= \sum_j \lim_n \nu_n^e(A_j) = \sum_j \langle e : \tau\rangle(A_j).\end{aligned}$$

By ????, the sequence $\langle e : \tau\rangle_n$ is monotonically increasing for values and error. Moreover, $\langle e : \tau\rangle_n$ defines The second equality is monotone convergence on $\mathbb{N}$ with counting measure. Also $\langle e : \tau\rangle(T_\tau) \leq 1$, since every ν_n^e is a subprobability measure, and thus $\langle e : \tau\rangle_n \leq 1$ for every $n \in \mathbb{N}$. Thus , by the monotone convergence theorem, the pointwise limit exists. Thus the limit is a subprobability measure.

For all $n \in \mathbb{N}$,

$\pi_1(\lim_{n\to\infty}\langle e, e'\rangle_n) = \lim_{n\to\infty}\pi_1(\langle e, e'\rangle_n)$

$\pi_2(\lim_{n\to\infty}\langle e, e'\rangle_n) = \lim_{n\to\infty}\pi_2(\langle e, e'\rangle_n)$

We prove part i) in detail; the proof of part ii) is analogous. Fix an arbitrary measurable$A \in \Sigma_1$: Since A was arbitrary, this implies: as desiredFor every fixed measurable terminal event, the n-step kernel lemmas make its probability a measurable function of the initial state. Its pointwise limit is measurable, which proves the sub-Markov-kernel claim. □

```
let x1 = uniform(0,1) in
let x2 = if x1 < 0.56 then uniform(0,2) else uniform(0,3) in
let x3 = if x2 < 0.92 then uniform(0,4) else uniform(0,5) in
let x4 = if x3 < 0.47 then uniform(0,6) else uniform(0,7) in
let x5 = if x4 < 0.51 then uniform(0,8) else uniform(0,9) in
x5 < 0.5
```

Fig. 28. Conditional Independent (Figure 36a): Program of size 5. Each variable depends on its immediate predecessor variable via a conditional branch. Each uniform distribution is distinct.

```
let x1 = uniform(0,1) in
let x2 = if x1 < 0.56 then uniform(0,2) else uniform(0,3) in
let x3 = if x1 < 0.45 then uniform(0,4) else uniform(0,5) in
let x4 = if x1 < 0.18 then uniform(0,6) else uniform(0,7) in
let x5 = if x1 < 0.48 then uniform(0,8) else uniform(0,9) in
x5 < 0.5
```

Fig. 29. Conditional Independent Fork (Figure 36b): Program of size 5. Each variable x_i depends on the same predecessor variable x_1 through a conditional branch. This creates a fork-shaped dependency pattern in which x_1 serves as the sole parent of all subsequent variables. Each uniform distribution is distinct.

```
let x1 = uniform(0,1) in
let x2 = if x1 < 0.18 then uniform(0,2) else uniform(0,3) in
let x3 = if x1 < 0.63 then uniform(0,4) else uniform(0,5) in
let x4 = if x2 < 0.09 then uniform(0,6) else uniform(0,7) in
let x5 = if x1 < 0.14 then uniform(0,8) else uniform(0,9) in
x5 < 0.5
```

Fig. 30. Conditional Random 1 (Figure 36c): Program of size 5. Each variable depends on a randomly selected predecessor via a conditional branch. Note that this structure may lead to unused variables; that is, some previously defined x_i may not influence any subsequent computation. Each uniform distribution is distinct.

No coupled finite-step or big-step transfer is claimed here. Such a result requires a measurable weak-simulation relation closed under generated-target administrative steps, an error-total coupling continuation, and a proved compatibility result between restriction to terminal outcomes and component pushforward. The exact-graph synchronous construction above does not supply these facts.

G Benchmarks

G.1 Asymptotic Scaling

G.1.1 Code Sketches for Scaling Benchmarks. This section contains code snippets of the programs used in the scaling experiments in Section 7.2. We implemented a Python script that automatically generates these programs based on configurable parameters such as program size, guard structure, and dependency pattern. For reproducibility, we used a fixed random seed to generate each program. All r_i values in the generated programs, where $0 \le r_i \le 1$ is randomly chosen, are rounded to two decimal places.

All of these programs follow the same structural pattern: they define a sequence of random variables $X_1, \ldots, X_n$, where each X_i is drawn from a uniform distribution whose parameters depend on the value of a single predecessor variable through a conditional branch of the form

$$X_i = \begin{cases} \mathrm{Unif}(a_i, b_i), & X_{\pi(i)} < r_i, \\ \mathrm{Unif}(a_i', b_i'), & \text{otherwise}, \end{cases}$$

for some parent-selection function $\pi : \{2, \ldots, n\} \to \{1, \ldots, i-1\}$. Thus each program is a Bayesian network of conditionally independent nodes, differing only in the choice of $\pi(i)$ that determines the dependency pattern.